
Ponder Engine: A Hippocampal Memory Architecture for AI Agents

A brain-inspired alternative to the context window that separates knowledge from processing, enables an artificial subconscious, and makes domain expertise a transferrable, composable resource.

Technical Architecture Proposal – v2.0, July 2026

Executive Summary

Current AI agents rely on **context windows** – fixed-size text buffers that concatenate conversation history and feed it to a language model. This is a von Neumann architecture solution to a biological problem: separate storage from processing, copy data between them, truncate when the buffer overflows. The brain never evolved this assumption. It stores patterns in the same neurons that process them, and "retrieval" is reactivation of those neurons via sparse hippocampal pointers.

We propose a **ponder engine** – a hippocampal memory architecture that replaces the context window with a brain-inspired alternative. The system encodes conversation, documents, code, and multimedia into structured triples stored in a graph-based hippocampal index over a distributed content store. Working memory is the set of currently activated patterns rather than a fixed text buffer. Retrieval is pattern completion via graph traversal. Consolidation occurs during offline "dream state" passes via a graph neural network. Forgetting is managed versioning, not destructive truncation.

Beneath conscious awareness, a **subconscious routing system** continuously decides which domain to query, which retrieval pathway to use, what model size is needed, and whether conscious deliberation is even required. This router learns from every interaction, becoming personalized over time.

The architecture is built on **WaveDB** – an existing hierarchical key-value database with MVCC concurrency, a Graph layer with Gremlin-style traversal, and production Python bindings – combined with small, specialized models for extraction, reasoning, and consolidation. The entire system runs on consumer hardware. Total training cost: ~\$120. Total inference cost: \$0.

1. Problem Statement

1.1 The Context Window Is the Wrong Abstraction

Every major AI assistant operates on the same fundamental model:

```
User query → [entire conversation history loaded into context window]
              → LLM processes via self-attention → response
```

This has three structural problems:

Problem Consequence

Fixed capacity When conversation exceeds the token limit, information is permanently lost via truncation, summarization, or sliding windows – all lossy hacks

$O(n^2)$ attention Every token attends to every other token. Computational cost grows quadratically with conversation length

No native memory The model has no persistent memory between sessions. RAG adds an explicit retrieval step, but it's bolted on, not integrated

1.2 The Brain Does It Differently

The brain does not "load" memories into a buffer. It stores a **sparse index** in the hippocampus – compressed pointers to the neocortical neurons that, when reactivated together, reconstruct the memory. This is **hippocampal indexing theory** (Teyler & DiScenna, 1986).

The retrieval process:

1. **A cue arrives** – a partial input (a face, a smell, a question)
2. **Pattern completion in CA3** – the hippocampal subfield CA3 is a recurrent autoassociative network. It takes the partial cue and fills in the missing components, reconstructing the full hippocampal index in ~100-200 milliseconds
3. **Neocortical reactivation** – the completed hippocampal index sends signals back to the neocortical sites it points to. The memory is **reconstructed**, not retrieved
4. **The reconstruction IS the short-term access** – there is no separate "working memory copy." Working memory is simply the subset of long-term memory currently activated by attention (the **embedded-processes model**, Cowan 1999, 2005)

The brain also has **complementary learning systems**: the hippocampus learns fast and separates patterns, while the neocortex learns slow and extracts statistical regularities. During sleep, the hippocampus replays memories to gradually train the neocortex without catastrophic interference. And beneath conscious awareness, the **basal ganglia** continuously route between processing pathways – a subconscious gating system that learns which cognitive mode to engage based on context.

1.3 What We're Building

A memory architecture for AI agents that mirrors this design, with three key innovations beyond biological mapping:

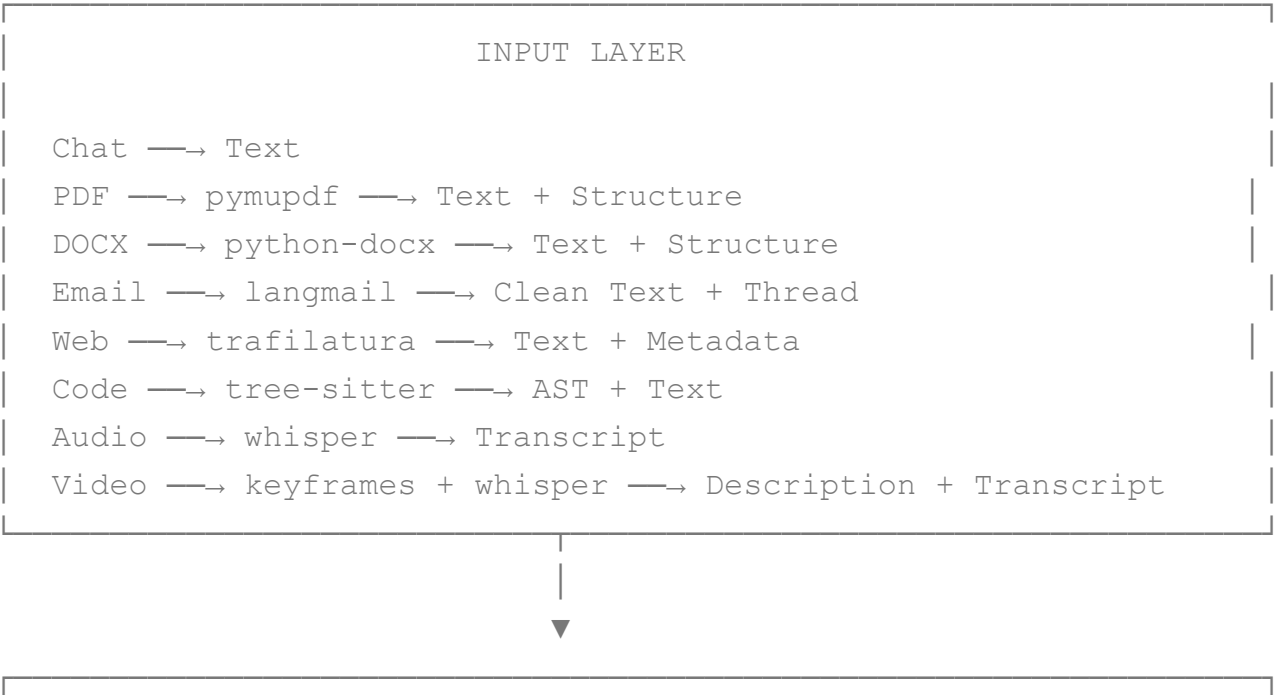
1. **A learned subconscious router** that decides which domain to query, which pathway to use, and what model size is needed – before any model generates a token
2. **Domain knowledge as transferrable graphs** – expertise that can be exported, imported, and composed without retraining any model
3. **A bidirectional interface** that lets any LLM consume curated context and write back memory commands

Brain Component Our Implementation

Neocortical store (distributed, slow-learning) WaveDB HBTree – hierarchical key-value store with MVCC
Hippocampal index (sparse, fast-learning) WaveDB Graph layer – triple store with SPO/POS/OSP indexes
CA3 pattern completion Graph traversal (Gremlin-style) from partial cues
Sharp-wave ripple replay GNN-driven consolidation during offline "dream state"
Working memory SSM (Mamba) continuous hidden state + activated graph nodes
Basal ganglia routing JEPA gate – learned subconscious router
Dopamine/salience modulation JEPA predictor + retrieval-weighted persistence
Reality monitoring Bonsai (ternary 8B model) waking during anomalies
Systems consolidation Morphism formation + subgraph abstraction into semantic memories
Procedural memory (basal ganglia/cerebellum) Stored processes with delegation rules

2. Architecture Overview

2.1 System Diagram



ENCODING PIPELINE

GLiNER-Decoder
(205M, CPU)

Open discovery:
Invents labels
freely. Feeds
ontology evol.

GLiNER2
(205M, CPU)

Stable extract:
Entities, topics,
tones, decisions
vs evolved schema

Bonsai
(8B ternary,
GPU, 2.15GB)

Relation
extraction:
"How do these
relate?"

Hierarchical Chunker + Cross-Reference Detector
Chunks documents by natural structure. Detects
citations, contradictions with existing knowledge.

Output: Structured triples + content



STORAGE LAYER (WaveDB)

GRAPH LAYER
(Hippocampal Index)

Triple store. SPO/POS/
OSP indexes. Gremlin
traversal. Morphisms.

(ep_047, has_entity,
E:Alice)

(ep_047, has_topic,
T:config)

(doc_023, cites,
doc_012)

(WaveDB, subclassOf,
Database)

HBTRIE
(Neocortical Store)

Hierarchical KV with
MVCC. Content-addressed.

ep/047/summary

ep/047/text

doc/023/section/003

code/auth.py@v3/content

media/img_047/raw

VECTOR INDEX (FAISS/Usearch)

Semantic similarity search. Complementary to graph.



SUBCONSCIOUS LAYER

JEPA-GATED SSM PRIMITIVE (shared backbone, ~480M, GPU)

Shared: Mamba SSM weights (~370M) + JEPA weights (~110M)

Per-instance: state vector (2KB) + gate (~500K) +
input/output projections (~100K)

Instances:

RETRIEVAL GATE

"Does SSM state cover this prompt?"

Routes: ssm_direct | graph_retrieve | process_exec
| tool_plan | conscious_deliberation

Predicts: domain(s), pathway, model size,
meta-skills needed, deliberation required

WORKING MEMORY

Continuous hidden state. No fixed token limit.

State evolves with each input. IS current awareness.

Retrieved memories injected as embeddings.

UNCERTAINTY DETECTOR

Flags: routing uncertainty, novel entities,
unresolved contradictions.

Feeds: EXPAND mechanism, delegation ladder.

ASPIRATIONAL MODEL

Present: encoding gate (JEPA salience)

Future: prospective memory (triggers)

Past: retrieval-weighted persistence

SELF-MODEL

Learned knowledge boundary thresholds.

"Do I know enough to answer this?"



RETRIEVAL LAYER

Bonsai Query
Planner

NL → structured
query parameters

Graph Traversal

Pattern
completion via
Gremlin traversal

Vector Search

Semantic
similarity

ONTOLOGY EXPANDER

Expands queries using class hierarchy:

"databases" → WaveDB, Postgres, SQLite (subclasses)



CONSCIOUS LAYER (System 2)

TOOL USE PLANNER + PROCESS EXECUTOR + DELEGATION LADDER

Engages when subconscious confidence is low.

Plans multi-step strategies. Executes stored processes.

Delegates steps to larger models when needed.

Delegation: 1B → 3B → 8B → 70B → 175B

GENERATION MODELS

Mode A: Any LLM receives context string

Mode B: SSM decoder generates from state vector

Bidirectional: LLM writes back memory commands



CONSOLIDATION LAYER (Dream State – Offline)

GNN CONSOLIDATOR (~200M, GPU)

- ❶ Saliency Scoring – structural importance
- ❷ Subgraph Summarization (DiffPool) – abstraction
- ❸ Link Prediction (GAE/SEAL) – implicit edges
- ❹ Anomaly Detection – pattern violations
- ❺ Ontology Refinement – taxonomy evolution

BONSAI VERIFIER (wakes on anomalies)

Ternary {-1, 0, +1} judgments on GNN-flagged patterns

FORGETTING SYSTEM

- Retrieval-weighted persistence (self-limiting)
- Reconsolidation counting + two-phase LTP
- Active forgetting (user-triggered deprecation)
- Reconsolidation (old facts versioned, not deleted)
- Pruning (low-saliency edges archived)
- MVCC preserves all versions for historical queries

TRAINING LAYER (Offline)

ORACLE (DeepSeek, large, cloud API)

Generates initial training data. Provides correction signal during gestational training. Used once per component, then components operate independently.

SELF-SUPERVISED LEARNING

System generates training data from its own operation:
retrieval outcomes, Bonsai verifications, user
corrections, delegation outcomes → training signals

DEVELOPMENTAL TRACKER

Each component: INFANT → CHILD → ADOLESCENT → ADULT
Transitions gated by measurable metrics.

2.2 Component Summary

Component Parameters Hardware Role Runs When

WaveDB Graph Layer N/A CPU/disk Hippocampal index – sparse pointers to neocortical patterns

Always

WaveDB HBTrie N/A CPU/disk Neocortical store – actual content (text, embeddings, structured data, media) Always

Vector Index (FAISS/Usearch) N/A CPU/GPU Semantic similarity search On retrieval

GLiNER-Decoder 205M CPU Open entity discovery – invents labels freely, feeds ontology evolution
Every turn

GLiNER2 205M CPU Stable extraction against evolved schema – entities, topics, tones, decisions
Every turn

Hierarchical Chunker N/A CPU Document structure preservation – chunks by headings, not token
windows On ingestion

Cross-Reference Detector N/A CPU Inter-document linking – citations, contradictions with existing
knowledge On ingestion

Bonsai 8B ternary (2.15 GB) CPU Relation extraction, query planning, anomaly verification
Every turn +
on wake

~~**JEPA-Gated SSM (shared backbone)** 480M GPU Shared dynamics for all cognitive instances. Mamba
SSM (370M) + JEPA predictor (~110M). Loaded once. Real-time~~

Retrieval Gate (instance) ~500K + state GPU Subconscious router. Predicts domain, pathway, model
size, meta-skills, deliberation need. Every prompt

Working Memory (instance) ~500K + state GPU Continuous hidden state. No fixed token limit. IS current
awareness. Real-time

Uncertainty Detector (instance) ~500K + state GPU Flags routing uncertainty, novel entities,
unresolved contradictions. Feeds EXPAND and delegation. Every prompt

Aspirational Model (instance) ~500K + state GPU Three temporal modes: encoding gate (present),

prospective memory (future), retrieval-weighted persistence (past) Every turn + continuous

Self-Model (instance) ~500K + state GPU Learned knowledge boundary thresholds. "Do I know enough?"

Every prompt

GNN Consolidator ~200M GPU Saliency scoring, subgraph summarization, link prediction, anomaly detection, ontology refinement Dream state (offline)

Forgetting System N/A CPU Retrieval-weighted persistence, reconsolidation counting, two-phase LTP, active forgetting, pruning Continuous + periodic

Process Executor N/A CPU Executes stored processes. Delegates steps to larger models. Handles failure modes. On process invocation

Delegation Ladder 1B→3B→8B→70B→175B Mixed Capability-appropriate intelligence. Each level handles what it can. On delegation

Developmental Tracker N/A CPU Tracks maturation stage per component. Gates transitions by metrics. Continuous

Oracle (DeepSeek) Large Cloud API Training signal for all learned components Gestational training

3. Component Justifications

3.1 WaveDB Graph Layer → Hippocampal Index

Why not a vector database? Vector similarity answers "what's similar to this?" but not "what did Alice say about databases last week?" or "what happened after we implemented morphisms?" The graph stores **discrete, symbolic relationships** — entity presence, topic classification, affective state, temporal order, decisions, contradictions, citations. These are different in kind from embedding similarity and answer different questions.

Why WaveDB specifically? WaveDB provides a triple store with SPO/POS/OSP indexes, Gremlin-style traversal, cost-based query optimization, and morphisms (reusable query fragments). The morphism system maps directly to **systems consolidation** — frequently traversed paths become single named pathways, analogous to how the neocortex learns direct connections after repeated hippocampal replay.

Key design decision: The graph stores **structure** (who, what, when, how felt, in what order). The HBTrie stores **content** (the actual words, code, media). The vector index stores **similarity**. Each answers different questions, and together they form a memory system no single component could provide.

3.2 WaveDB HBTrie → Neocortical Store

Why hierarchical key-value? The HBTrie's hierarchical paths (`episodes/ep_47/summary` , `entities/alice/properties/name` , `code/auth.py@v3/content`) map naturally to

distributed cortical representations. Each level is a "cortical region" — same underlying storage, different organizational principles.

Why MVCC? Multi-Version Concurrency Control provides snapshot isolation — every read sees a consistent version. This maps to **state-dependent memory**: the same memory can have different "versions" depending on when you access it. Combined with edge validity windows in the Graph layer, this enables temporal queries ("what did I know at time T?") without losing historical versions.

3.3 GLiNER-Decoder → Open Entity Discovery

Why not just use a predefined schema? A static schema is a filter — it tells the system what to see, which means it also tells it what to ignore. The brain discovers categories through experience. GLiNER-Decoder (Knowledgator, 205M parameters) uses a small generative decoder to **invent labels** for discovered entities rather than matching against a predefined list. Given `labels=["label"]`, it produces spans with freely generated type names.

Why it matters: The system discovers that this user talks about `deployment`, `testing`, and `documentation` — categories no one predicted. These discovered labels are clustered, canonicalized, and promoted to the stable schema over time. The schema is alive.

3.4 GLiNER2 → Stable Extraction

Why two GLiNER variants? GLiNER-Decoder is flexible but inconsistent — the same entity might be labeled `"configuration option"` in one conversation and `"setting"` in another. GLiNER2 (Fastino, 205M parameters) extracts against a stable, evolved schema with consistent labels. It's faster and more reliable for categories that have been promoted from discovery.

Why 205M parameters? A 459M GLiNER encoder outperformed ChatGPT (~175B) on named entity recognition benchmarks. For extraction tasks, specialized small models beat generalist large models. GLiNER2 runs on CPU, costs nothing per inference, and produces structured output in a single forward pass.

3.5 Bonsai → Relation Reasoning, Query Planning, Verification

Why an 8B ternary model? Three distinct roles require reasoning beyond what GLiNER can do:

1. **Relation extraction:** GLiNER finds entities. Bonsai determines how they relate — "Alice explained DEBOUNCED," "User decided to use HBtrie," "This contradicts the earlier decision." This requires understanding semantic content, not just spotting named entities.
2. **Query planning:** Converting "What was I frustrated about last week?" into `{tones: ["frustrated"], temporal_filter: "last_week", entity_mode: "union"}`

requires reasoning about query intent. GLiNER can extract entities from the prompt, but it can't determine that "Alice and I decided" means entity_mode should be "union" (episodes involving Alice) rather than "intersection" (episodes containing both Alice AND the user).

3. **Reality monitoring:** During consolidation, Bonsai wakes to verify GNN-flagged anomalies. The ternary weights $\{-1, 0, +1\}$ map to inhibitory/excitatory/absent synapses — the model can actively suppress incoherent reconstructions, not just fail to excite them.

Why ternary? The 8B ternary model occupies ~2.15 GB — it fits on an RTX 5060 Ti (16 GB) alongside the SSM and JEPA. Ternary weights also map to biological synapse states: potentiated (+1), depressed (-1), or silent/absent (0).

3.6 JEPA-Gated SSM → Unified Cognitive Primitive

Why a shared backbone? The previous architecture treated the Mamba SSM, JEPA Predictor, and Gate as separate models with separate purposes. This had three consequences: proliferation of models (each new cognitive function required a new model), no unified dynamics (components were independent when they should be aspects of a single primitive), and missing cognitive functions (no disturbance detection, no prospective memory, no developmental staging).

The JEPA-gated SSM primitive solves this through a critical distinction: **shared weights, separate states**. The SSM and JEPA weights are loaded once in GPU memory (~~480M parameters, ~0.90 GB~~). ~~Every cognitive function is an instance with its own state vector (512 floats, ~2 KB), its own gate (500K parameters), and its own input/output projections (~100K parameters).~~ Adding a new cognitive function costs ~1M parameters, not ~500M.

Why this doesn't collapse: A single state vector cannot simultaneously represent retrieval gating, working memory maintenance, uncertainty detection, aspiration, and self-modeling. These are different kinds of information with different temporal dynamics. The architecture prevents collapse through separate state spaces (each instance has its own 512-dimensional vector), input/output projections (learned lenses that map instance-specific inputs into the shared representational space), and instance-specific gates (each learns its own inhibit/excite policy).

Biological mapping: This maps onto the canonical cortical microcircuit — the same cell types, laminar structure, and local connectivity repeated across the entire neocortex. What differs between regions is input sources, output targets, and inhibitory interneuron composition. The circuit is the same. The configuration makes it different.

3.7 Retrieval Gate → Subconscious Router

Why learned, not rule-based? The decision of which domain to query, which pathway to use, and what model size is needed depends on nuanced factors: how recently was this entity discussed? How

confident is the SSM about this topic? Is this a follow-up question or a topic shift? Is this pattern similar to past queries that required delegation? A learned gate learns these patterns from experience.

What it predicts (before any model generates a token):

- **Domain routing:** Which domain graph(s) to query – database, robotics, economics, coding, personal
- **Pathway selection:** ssm_direct | graph_retrieve | process_exec | tool_plan | conscious_deliberation
- **Meta-skill prediction:** What cognitive capabilities are needed – factual_recall, basic_synthesis, pattern_recognition, creative_synthesis, security_analysis
- **Model sizing:** 1B | 3B | 8B | 70B | 175B
- **Deliberation gating:** Can the subconscious handle this, or does System 2 need to engage?

How it learns: Every routing decision produces an outcome. Did the user accept the response? Did the model have to delegate unexpectedly? Was expansion needed? These outcomes feed back as reinforcement signals. Successful routes are reinforced. Routes that required unexpected delegation are penalized. Routes that used a larger model than necessary get an efficiency penalty. Over time, the gate becomes personalized – it learns your domains, your patterns, your habits.

3.8 Working Memory → Continuous Awareness

Why not a transformer? Transformers have $O(n^2)$ attention – the thing we're trying to escape. State Space Models (Mamba) maintain a continuous hidden state that evolves with each input. There is no fixed context window. Old information decays gracefully rather than being truncated. The state dimension is constant, but the information it represents expands and contracts dynamically.

Why it's working memory: The SSM state IS the currently activated subset of everything the system knows. When the retrieval gate determines the state is sufficient, no graph retrieval is needed – the SSM state alone generates the response. When the gate detects low confidence, it triggers graph retrieval and the retrieved content is injected into the SSM state as embeddings, not text.

3.9 Uncertainty Detector → Knowing When You Don't Know

Why it matters: Without uncertainty detection, the system either confabulates (generates from insufficient context) or retrieves unnecessarily (wastes computation on queries the SSM state already covers). The Uncertainty Detector flags three conditions:

1. **Routing uncertainty:** JEPA confidence is below threshold for the routed domain
2. **Novel entities:** Retrieved context contains entities not in the ontology
3. **Unresolved contradictions:** Multiple sources contradict without resolution

These flags feed the EXPAND mechanism (load more detail from HBTree) and the delegation ladder (route to a larger model). The detector learns its thresholds from experience – during gestational training, the Oracle provides feedback on when the system should have known it didn't know.

3.10 Aspirational Model → Three Temporal Modes

Why three modes? Wanting operates in all three temporal directions. The same dopaminergic signal that gates encoding also maintains goal representations in working memory (prospective intention) and tags memories for replay priority during consolidation. The Aspirational Model reflects this unity:

- **Present (encoding gate):** Modulates encoding strength based on aspirational match. "I'm curious about this → encode strongly." Implemented through JEPA salience scoring.
- **Future (prospective memory):** Sets triggers that fire when conditions are met. "Alert me when you encounter information about X." The system checks pending triggers before each retrieval loop.
- **Past (retrieval-weighted persistence):** Every retrieval boosts the edge's long-term survival probability. The more the user asks, the harder it is to forget – with self-limiting mechanisms to prevent immortal memories.

3.11 Self-Model → Knowledge Boundary Estimation

Why learned thresholds? The system needs to know when to say "I don't know." Hardcoded thresholds ("if graph density < 0.3, admit gap") fail because different domains have different knowledge densities. A sparse but complete domain (3 episodes about a niche topic) should produce confident answers. A dense but incomplete domain (100 episodes but missing the specific fact) should not. The Self-Model learns these boundaries from Oracle feedback during gestational training.

3.12 GNN Consolidator → Dream State Processing

Why a graph neural network? The consolidation tasks – salience scoring, subgraph summarization, link prediction, anomaly detection, ontology refinement – all require reasoning about **graph structure**, not just node features. A node's importance depends on its position in the graph (bridge, hub, peripheral), not just its content. GNNs are the standard tool for this class of problem.

Why offline? Consolidation is computationally intensive and benefits from seeing the full graph. Running it during the dream state (offline, periodic) mirrors the brain's sharp-wave ripple replay during sleep. The GNN identifies clusters for abstraction, predicts missing edges, detects anomalies, and refines the ontology – then Bonsai verifies the high-stakes decisions.

Why static first, temporal later? The initial GNN is a stateless function: `consolidate(graph) → actions`. After collecting data on failure modes (cluster flapping, prediction miscalibration, false positive anomalies, ontology oscillation), we add SSM-augmented instances that remember past

decisions. This prevents premature optimization — we build temporal continuity only for the failure modes that actually occur.

3.13 Forgetting System → Self-Limiting Persistence

Why not flat decay? Flat utility score decay treats all edges identically. But edges that are repeatedly retrieved should persist longer — not indefinitely, but proportionally to demonstrated importance. The forgetting system combines four mechanisms:

1. **Retrieval-weighted persistence:** Each retrieval reduces the edge's decay rate. Strong matches (direct entity+topic+tone match) reduce it more than weak matches (ontology expansion). Diminishing returns prevent immortal memories — the 20th retrieval provides far less boost than the 1st.
2. **Saturation detection:** If the same edge is retrieved more than N times in 24 hours, the system stops boosting and may slightly increase decay. This breaks the "frustration loop" — the user keeps asking because the answer isn't sticking, and the system keeps reinforcing because the user keeps asking.
3. **LLM-mediated importance signal:** The LLM embeds signals in its response — `[ IMPORTANT ]` , `[ ROUTINE ]` , `[ FRUSTRATION ]` , `[ CORRECTION ]` . These modulate the persistence boost. A frustration signal prevents reinforcement. A correction signal may increase decay on the old fact.
4. **Boost decay:** The persistence boost from a retrieval itself decays with a ~7-day half-life. A retrieval last month matters less than a retrieval yesterday. If the user stops asking, the boost fades and the edge returns to baseline decay.

Reconsolidation counting: Edges track how many times they've been retrieved. Three retrievals across 15+ days triggers late-phase LTP — a 70% reduction in decay rate on top of accumulated boosts. This is the architectural equivalent of the transition from hippocampal-dependent to neocortical-independent memory.

3.14 Process Executor → Procedural Memory

Why stored processes? When the system observes the user performing the same multi-step task three times, it proposes a stored process. The fourth time, the user says "review this" and the system executes the process automatically, delegating individual steps to larger models when they exceed the current model's capability.

Processes are stored in the same graph as everything else — they're just subgraphs with a specific shape. Each step specifies what tool to use, what model size is needed, and what to do on failure (delegate, skip, ask_user, abort). Meta-processes — strategy templates with decision points rather than fixed steps — guide the model's reasoning when no concrete process matches.

3.15 Delegation Ladder → Capability-Appropriate Intelligence

Why delegate? A 1B model can summarize. A 3B model can synthesize. An 8B model can reason. A 70B model can design. A 175B model can create. The delegation ladder routes each step of a process to the smallest model that can handle it, escalating only when necessary. The user never knows a larger model was invoked.

The Retrieval Gate learns which model size is needed for which patterns. Over time, it routes directly to the right size without trial and error. Most real-world queries (80%) are handled by the 1-3B models. The 70B+ models sit idle until genuinely needed.

3.16 Oracle (DeepSeek) → Gestational Training

Why an Oracle? Several components require training data that doesn't exist in public datasets: query planning pairs, relation extraction from conversational text, salience labels on memory graphs, gate training examples. The Oracle generates this data during a gestational training phase.

Why not permanent? The Oracle is expensive per call. It generates 5,000-10,000 training examples per component during initial training, then the components operate independently. The Oracle is only invoked again if the system encounters genuinely novel situations that the fine-tuned models can't handle. The Developmental Tracker gates this – components graduate from INFANT (Oracle provides all signal) to CHILD (Oracle critiques decisions) to ADOLESCENT (self-regulating) to ADULT (independent).

4. Information Flow

4.1 Encoding Flow (Every Conversation Turn)

1. Raw conversation text enters the system
2. GLiNER-Decoder performs open discovery:
"WAL config" → "technical concept"
"DEBOUNCED" → "configuration option"
"confusing" → "sentiment"
"go with DEBOUNCED" → "decision"
→ Raw entity spans with invented type labels
3. GLiNER2 performs stable extraction against the evolved schema:
entities: ["Alice", "WaveDB"]
topics: ["configuration"]
tones: ["frustrated", "curious"]

decisions: ["use_debounced"]
→ Structured, consistent labels

4. Discovered labels are buffered. Labels appearing >10 times are clustered, canonicalized, and promoted to the stable schema.
"configuration option" + "setting" + "parameter" → "configuration"
5. Bonsai extracts relations:
(User, expresses, frustrated)
(User, questions, WAL_sync_modes)
(Alice, explains, WAL_sync_modes)
(User, decides, use_debounced)
(conversation, concerns, configuration)
(conversation, involves, Alice)
6. Aspirational Model (present mode) scores encoding strength:
salience = 0.65 (moderate – contains a decision but routine topic)
→ Encode at baseline strength
7. Storage:
Graph layer: insert all triples as edges
HBtrie: store summary, full text, timestamp
Vector index: store embedding of summary for semantic search
8. Process Observer checks for repeated patterns:
"User asked about config → assistant explained → user decided"
Seen 3 times? → Propose stored process

4.2 Retrieval Flow (On User Prompt)

1. User prompt: "What was I frustrated about last week?"
2. Aspirational Model (future mode) checks pending triggers:
Any prospective memories matching this context?
→ None found. Continue.
3. Retrieval Gate (subconscious router) evaluates:
 - "frustrated" tone not in recent SSM state → low confidence
 - "last week" temporal scope → SSM state only covers recent turns
 - Domain: database (confidence 0.89)
 - Pathway: graph_retrieve
 - Model size: 3B (factual_recall + basic_synthesis)

- Deliberation: NOT NEEDED
- TRIGGER GRAPH RETRIEVAL

4. Bonsai plans the query:

```
{  
  entities: [],  
  topics: [],  
  tones: ["frustrated"],  
  entity_mode: "union",  
  temporal_filter: "last_week",  
  limit: 5  
}
```

5. Ontology Expander expands the query:

- "frustrated" → Frustrated (subclass of AffectiveTone)
- Also match episodes with tone "Frustrated"

6. Graph traversal (pattern completion):

- Find all episodes with tone "frustrated" (union)
- Filter by timestamp > 7 days ago
- Score by: tone match count + recency + retrieval history
- Return top 5

7. HBtrie load (neocortical reactivation):

- For each result episode ID:
 - load summary, full_text, timestamp from HBtrie

8. Working memory = activated graph nodes + HBtrie values + SSM state

9. Uncertainty Detector checks:

- Routing confidence: 0.89 → OK
- Novel entities in results: none → OK
- Unresolved contradictions: none → OK
- No flags. Proceed.

10. Response generation from working memory

11. Retrieval-weighted persistence:

- For each edge traversed in retrieval:
 - on_retrieve(retrieval_strength, llm_signal)
 - Decay rate reduced (diminishing returns)
 - Saturation check passed
 - Reconsolidation count incremented

4.3 Consolidation Flow (Dream State – Periodic)

1. GNN scores all nodes/edges by structural salience:
 - ep_001: 0.92 (anchor episode, foundational decision)
 - ep_005: 0.30 (routine, redundant paths exist)
 - edge ep_003_has_entity_Bob: 0.80 (only connection for Bob)
 - Salience incorporates: access_count, reconsolidation_count, ltp_phase
2. GNN detects subgraphs ready for abstraction:
 - Cluster: [ep_001, ep_002, ep_003, ep_004]
 - Abstract to: "WaveDB initial development (June 20-23)"
 - Store semantic memory in HBtrie
 - Link semantic memory to source episodes
 - Mark source episodes as "abstracted" (still retrievable)
3. GNN predicts missing edges:
 - (Postgres, related_to, performance) confidence 0.82
 - Auto-accept (confidence > 0.8)
 - (ep_006, caused_by, ep_005) confidence 0.75
 - Propose for Bonsai verification (confidence 0.5-0.8)
4. GNN detects anomalies:
 - MISSING_TEMPORAL: ep_007 follows ep_004, but ep_005-006 also follow ep_004
 - Wake Bonsai: "Are these parallel threads?"
 - DUPLICATE_DECISION: use_hbtrie (ep_001) and use_wavedb_over_postgres (ep_003)
 - Wake Bonsai: "Are these the same decision?"
5. Bonsai verifies anomalies:
 - Temporal branch: "Legitimate parallel threads. No change needed."
 - Duplicate decision: "Distinct decisions. Add 'reinforces' edge."
6. Forgetting pass:
 - Compute effective decay rate for each edge (incorporates retrieval history)
 - Edges with utility_score < 0.1 and state="current" → state="archived"
 - Superseded edges older than 90 days → compress to summary, archive
 - Boost decay: old retrieval boosts fade (half-life ~7 days)
 - Ontology categories not seen in 30+ days → deprecate

7. SSM replay:

- Replay high-salience consolidated memories through SSM
- Strengthens neocortical (SSM weight) representations

8. Developmental Tracker evaluates:

- Extraction F1 > 0.85? → Graduate from INFANT to CHILD
- Retrieval recall > 0.80? → Graduate from INFANT to CHILD
- Gate decisions match Oracle > 0.75? → Graduate from INFANT to CHILD

4.4 Forgetting Flow (Continuous + Periodic)

1. RETRIEVAL-WEIGHTED PERSISTENCE (on every retrieval):

- Diminishing returns: each retrieval boosts less than the previous
- Saturation detection: >N retrievals in 24 hours → stop boosting
- LLM-mediated signal: [IMPORTANT] → stronger boost, [FRUSTRATION] → no boost
- Boost decay: old retrieval boosts fade with ~7-day half-life
- Absolute floor: decay rate never reaches zero (minimum 0.1%/day)

2. RECONSOLIDATION COUNTING (on every retrieval):

- Track reconsolidation_count and consolidation_window_start
- 3 retrievals across 15+ days → late-phase LTP
- Late-phase LTP: additional 70% reduction in decay rate

3. ACTIVE FORGETTING (user-triggered):

- User: "Forget what I said about Postgres"
- Edge state → "deprecated"
- validity_end → now()
- Edge still exists, not returned by default queries
- Retrievable via historical query: "what did I used to think?"

4. RECONSOLIDATION (on contradiction):

- New fact contradicts old fact
- Old fact: state → "superseded", validity_end → now()
- New fact: state → "current", validity_start → now()
- Link: new_fact supersedes old_fact, new_fact contradicts old_fact
- Both versions preserved. Default queries return current only.

5. PRUNING (periodic consolidation):

- Archived edges older than 365 days → deep archive

- Superseded edges → compressed to summary in HBtrie
 - Nothing is ever truly deleted. Storage is cheap.
-

5. The Evolving Ontology

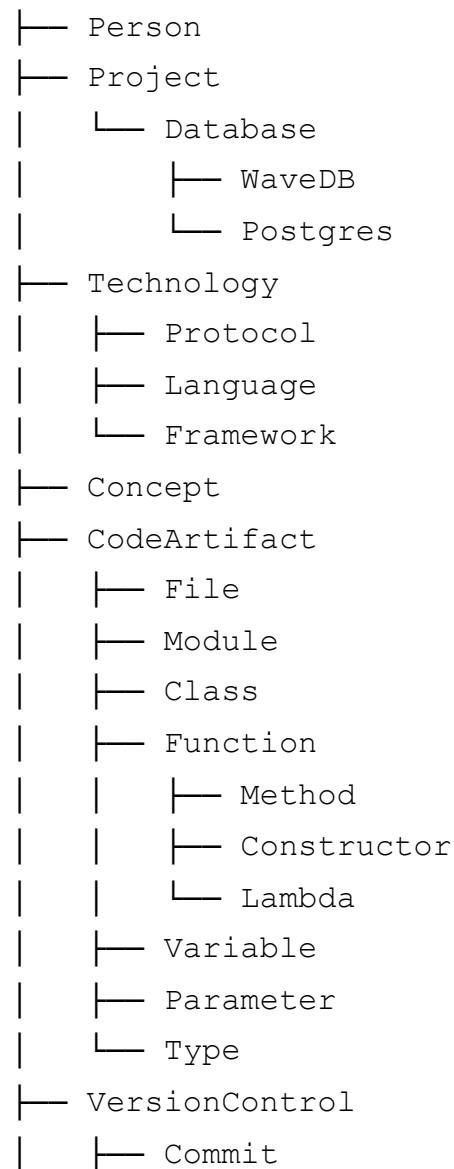
5.1 Why an Ontology?

Without hierarchical knowledge, the system can't answer "What databases did we discuss?" — it would only match episodes explicitly labeled "database," missing episodes about WaveDB, Postgres, or SQLite. With a taxonomy where WaveDB IS-A Database, the query expands automatically.

5.2 Seed Ontology

The seed ontology includes both conversational and code entity types. This ensures the models allocate representational capacity for code structure from the start, even before any code is parsed.

Entity



- | | — Branch
- | | — PullRequest
- | | — Release
- | — Issue
- | | — Bug
- | | — Feature
- | | — Regression
- | — Event
 - | — Decision
 - | — Explanation
 - | — Question

AffectiveTone

- | — Frustrated
- | — Excited
- | — Curious
- | — Neutral

Topic

- | — DatabaseDesign
- | — Configuration
- | — Performance
- | — Security
- | — APIDesign
- | — AIArchitecture
- | — DecisionMaking

5.3 How It Evolves

DISCOVERY (ongoing):

GLiNER-Decoder invents label "deployment strategy"

→ Buffered. After 10 occurrences:

→ Bonsai entailment: "Is 'deployment strategy' a Concept?"

→ Yes → added to ontology: Concept → DeploymentStrategy

REFINEMENT (periodic GNN pass):

GNN predicts: (DEBOUNCED, subclassOf, WALSyncMode) confidence 0.90

→ New class WALSyncMode created

→ DEBOUNCED, IMMEDIATE, ASYNC become subclasses

DECAY (periodic):

Category "Framework" not seen in 45 days

→ Deprecated. Entities reassigned to parent "Technology"

6. Memory Types Stored in the Graph

Memory Type Graph Structure Example

Episode (ep_047, has_entity, E:Alice), (ep_047, has_topic, T:config), (ep_047, has_tone, A:frustrated), (ep_047, follows, ep_046) One complete conversational exchange

Document (doc_023, type, Document), (doc_023, has_section, sec_003), (doc_023, cites, doc_012), (doc_023, contradicts, ep_012) An ingested document with hierarchical sections

Code (auth.py, contains, authenticate), (authenticate, calls, validate_token), (auth.py@v3, supersedes, auth.py@v2) Code structure and version history

Media (img_047, depicts, E:Alice), (img_047, has_tone, A:frustrated), (desc_img_047_v2, supersedes, desc_img_047_v1) Images, audio, video with versioned descriptions

Semantic Memory (sem_001, abstracts, ep_001), (sem_001, abstracts, ep_002), (sem_001, summary, "WaveDB development...") Consolidated abstraction of multiple episodes

Process (proc_review, has_step, step_1), (step_2, delegate_to, "70B"), (step_2, on_failure, "delegate") Stored procedure with steps and delegation rules

Meta-Process (meta_decompose, strategy_question, "Are sub-goals independent?"), (meta_decompose, if_yes, "parallel_decomposition") Strategy template with decision points

Ontology (WaveDB, subClassOf, Database), (Database, subClassOf, Project), (Frustrated, subClassOf, AffectiveTone) Class hierarchy for query expansion

Cross-Reference (doc_023, cites, doc_012), (doc_023, contradicts, ep_012), (TeddyVision, uses_for_logging, WaveDB) Links between documents, episodes, and domains

7. Implementation Plan

Phase 1: Bootstrap Extraction (Weeks 1-2)

Goal: All extraction components operational with acceptable accuracy. Training data generator for downstream phases.

Task Data Hardware Est. Time

GLiNER2 fine-tune on kniv-corpus-en 15K conversation examples with 18 NER types + 9 dialog act labels RTX 4090 cloud (\$1.08) 4 hours

GLiNER-Decoder evaluation on Memory-Traces 27K human-LLM conversations with salience/emotion labels CPU (local) 2 hours

Bonsai relation extraction fine-tune Oracle-generated pairs (2K examples) A100 cloud (\$6.56) 4 hours

WaveDB schema setup Graph layer schema + seed ontology (conversation + code) Local 1 day

Sample conversations 20 hand-crafted conversations with expected extraction labels Local 1 day

Corpus processing script Process DialogSum + SAMSum through encoding pipeline Local 1 day

Deliverable: System can encode conversations into structured triples with entities, topics, tones, decisions, and relations. Extraction quality: entity recall >70%, topic recall >70%, tone recall >70% on sample conversations. Corpus processing pipeline operational.

Phase 2: Train Shared Backbone + Retrieval Gate (Weeks 3-4)

Goal: JEPA-gated SSM shared backbone trained. Retrieval Gate instance operational. Mode A generation working.

Task Data Hardware Est. Time

Shared SSM+JEPA backbone training Oracle-generated cognitive state sequences (10M+ examples) A100 cloud (\$61.92) 48 hours

Retrieval Gate training Oracle-generated routing pairs (5K examples) RTX 4090 cloud (\$6.48) 24 hours

Working Memory instance setup SSM state initialization + input/output projections Local 1 day

Bonsai query planning fine-tune Oracle-generated pairs (5K examples) A100 cloud (\$16.40) 10 hours

Mode A generation integration Context window adapter for external LLMs Local 2 days

Integration testing Manual conversation scenarios Local 2 days

Deliverable: System can retrieve relevant memories given natural language questions. Retrieval Gate routes to correct domain and pathway. Mode A generation produces responses using retrieved context. JEPA correctly gates between ssm_direct and graph_retrieve pathways.

Phase 3: GNN Consolidator + Forgetting System (Weeks 5-6)

Goal: Dream state consolidation operational. Forgetting system with retrieval-weighted persistence.

Task Data Hardware Est. Time

GNN architecture implementation GAT + DiffPool + GAE (static, 5 task-specific heads) RTX 4090 cloud (\$4.32) 16 hours

Initial GNN training Oracle-generated salience labels + link prediction examples RTX 4090 cloud (\$8.64)

32 hours

Reconsolidation counting Edge model update: reconsolidation_count, ltp_phase, consolidation_window_start Local 1 day

Retrieval-weighted persistence Edge model update: retrieval_timestamps, saturation_flags, base_decay_rate, boost decay Local 1 day

LLM-mediated signal parsing Response parser for [IMPORTANT], [ROUTINE], [FRUSTRATION], [CORRECTION] Local 1 day

Consolidation loop integration WaveDB graph → GNN → Bonsai verify → WaveDB update Local 3 days

Deliverable: Nightly consolidation pass that scores salience, abstracts subgraphs, predicts links, and detects anomalies. Forgetting system with self-limiting retrieval-weighted persistence.

Reconsolidation counting with two-phase LTP.

Phase 4: Instance-Specific Gates (Weeks 7-8)

Goal: Uncertainty Detector, Aspirational Model, and Self-Model gates operational.

Task Data Hardware Est. Time

Uncertainty Detector gate training Oracle-labeled "is this a gap?" pairs (50K examples) RTX 4090 cloud (\$2.70) 10 hours

Aspirational Model gate training Oracle-labeled "should I commit?" pairs (50K examples) RTX 4090 cloud (\$2.70) 10 hours

Self-Model gate training Oracle-labeled "should I say I don't know?" pairs (50K examples) RTX 4090 cloud (\$2.70) 10 hours

Prospective memory implementation Trigger storage, condition matching, pre-retrieval check Local 2 days

EXPAND mechanism integration Uncertainty Detector → EXPAND → HBTree load → SSM injection Local 2 days

Developmental Tracker Stage tracking per component, metric evaluation, transition gating Local 1 day

Deliverable: System detects when it doesn't know, triggers EXPAND for missing details, delegates to larger models when needed. Prospective memory operational. Each component tracks its developmental stage.

Phase 5: Evaluation & Iteration (Weeks 9-10)

Goal: Quantify performance against benchmarks.

Benchmark What It Tests Target

ConvoMem 75K QA pairs, 6 evidence categories, 100 personas >80% recall on single-session, >60% on cross-session

EverMemBench Multi-party, 250-day span, 1M+ tokens >40% on multi-hop (baseline is 26% with oracle)

evidence)

CloneMem 1-3 year temporal reasoning >50% on temporal comparison

Phase 6: Process Learning + Delegation Ladder (Weeks 11-12)

Goal: Procedural memory and automatic delegation operational.

Task Data Hardware Est. Time

Process Observer Pattern detection in tool use sequences Local 2 days

Process Executor Step execution, dependency resolution, failure handling Local 3 days

Delegation Ladder Model registry, capability matrix, automatic escalation Local 2 days

Meta-process library Decomposition strategies, adaptation patterns, synthesis templates Local 2 days

Deliverable: System observes repeated task patterns and proposes stored processes. Executes processes with automatic delegation. Small models conduct larger models through process execution.

Phase 7: Self-Generated Training (Ongoing)

Goal: System improves from its own operation.

- Retrieval successes → positive salience signal for edges used
- Retrieval failures → training examples for missing edges
- Bonsai verifications → training examples for GNN anomaly detector
- User corrections → reconsolidation training examples
- Delegation outcomes → Retrieval Gate routing updates
- New entity types discovered → ontology refinement examples
- Process execution outcomes → process success rate updates

8. Cost Analysis

8.1 Cloud Training Costs

Phase Component GPU Provider Hours Cost

1 GLiNER2 fine-tune RTX 4090 Vast.ai spot 4 \$1.08

1 Bonsai relations (Oracle gen) A100 80GB RunPod Community 4 \$6.56

2 Shared SSM+JEPA backbone A100 80GB Lambda Labs 48 \$61.92

2 Retrieval Gate training RTX 4090 Vast.ai spot 24 \$6.48

2 Bonsai query planning A100 80GB RunPod Community 10 \$16.40

3 GNN implementation RTX 4090 Vast.ai spot 16 \$4.32

3 GNN training RTX 4090 Vast.ai spot 32 \$8.64

4 Uncertainty Detector gate RTX 4090 Vast.ai spot 10 \$2.70

4 Aspirational Model gate RTX 4090 Vast.ai spot 10 \$2.70
4 Self-Model gate RTX 4090 Vast.ai spot 10 \$2.70

TOTAL 168 \$113.50

8.2 Oracle API Costs

Use Tokens (est.) Cost (DeepSeek)

Generate 5K query planning pairs 2.5M input + 1M output → \$3.50
~~Generate 2K relation extraction examples 1M input + 0.5M output → \$1.75~~
~~Generate 10M+ cognitive state sequences (shared backbone) 5M input + 2M output → \$7.00~~
~~Generate 5K routing pairs (Retrieval Gate) 2.5M input + 1M output → \$3.50~~
~~Generate 1K salience labels (GNN) 0.5M input + 0.3M output → \$1.00~~
~~Generate 500 ontology refinement examples 0.3M input + 0.2M output → \$0.70~~
~~Generate 50K gate training examples (3 gates) 1.5M input + 0.8M output → \$2.50~~
TOTAL **\$20.00**

8.3 Local Hardware (Already Available)

Component Hardware Status

GLiNER2 + GLiNER-Decoder inference CPU  Runs on any modern CPU
Bonsai inference (8B ternary) RTX 5060 Ti (16 GB)  2.15 GB fits comfortably
~~JEPA Gated SSM shared backbone (480M) RTX 5060 Ti (16 GB)  0.90 GB~~
~~5x instance gates + states (5M total) RTX 5060 Ti (16 GB)  0.02 GB~~
~~GNN inference (200M) RTX 5060 Ti (16 GB)  ~0.40 GB~~
WaveDB CPU/disk  Production-ready
Vector Index CPU 

Total GPU VRAM usage: ~3.5 GB. Headroom: ~12.5 GB on RTX 5060 Ti.

8.4 Total Cost Summary

Category Cost

Cloud GPU training (all phases) \$114
~~Oracle API (DeepSeek) \$20~~
~~Local hardware \$0 (already owned)~~
~~WaveDB Free (MIT license, self-hosted)~~
GRAND TOTAL **\$134**

9. Key Design Decisions & Tradeoffs

9.1 Why Not a Single Large Model?

A single large model that does extraction, retrieval, and generation would be simpler. But:

- **Cost:** Running a 70B+ model for every conversation turn is expensive. GLiNER2 runs on CPU and costs nothing per inference. The delegation ladder only invokes large models when needed.
- **Specialization:** A 459M GLiNER encoder outperformed ChatGPT (~175B) on NER. Specialized small models beat generalist large models on their specific tasks.
- **Interpretability:** When retrieval fails, you can inspect the graph traversal to see why. A large model's retrieval is a black box.
- **Updateability:** You can fine-tune GLiNER2 on new entity types without retraining the entire system.

9.2 Why Not Pure Vector Search?

Vector search answers "what's similar to this embedding?" It doesn't answer:

- "What did Alice say about databases?" (entity + topic constraint)
- "What happened after we implemented morphisms?" (temporal chain)
- "What decisions did we make and why?" (relation type constraint)
- "What was I frustrated about?" (affective state constraint)

The graph stores these as discrete, queryable relationships. Vector search is complementary — it handles the "similar to this" queries that the graph can't.

9.3 Why Offline Consolidation?

Running the GNN on every conversation turn would add latency and computational cost. The brain doesn't consolidate in real-time either — it replays memories during sleep. The offline "dream state" pass allows the GNN to see the full graph, make global decisions about salience and abstraction, and not compete with real-time inference for GPU resources.

9.4 Why Never Delete?

Storage is cheap. The value of being able to answer "what did I used to think?" or "what changed and when?" outweighs the cost of keeping superseded edges. MVCC in WaveDB already versions every write. Edge validity windows add temporal semantics. The combination gives you current, historical, and change views without destructive operations.

9.5 Why Shared Weights, Separate States?

A single SSM state vector cannot simultaneously represent retrieval gating, working memory maintenance, uncertainty detection, aspiration, and self-modeling. These are different kinds of information with different temporal dynamics. The JEPA-gated SSM primitive prevents collapse through separate state spaces while sharing the expensive dynamics (SSM + JEPA weights). Adding a new cognitive function costs ~1M parameters, not ~500M.

9.6 Why Self-Limiting Persistence?

Positive-only reinforcement creates immortal memories. The user keeps asking about WAL config — is it because it's important, or because the answer isn't sticking? The self-limiting mechanisms (diminishing returns, saturation detection, LLM-mediated signals, boost decay) ensure that retrieval-weighted persistence reflects genuine importance, not frustration loops.

10. Success Metrics

Metric Target Measurement

Encoding accuracy >90% entity F1, >85% relation F1 Against Oracle-generated ground truth

Retrieval recall >80% on ConvoMem single-session, >60% cross-session Standard benchmark evaluation

Retrieval precision >85% Retrieved episodes contain answer

Retrieval latency <50ms for graph traversal + HBTrie load WaveDB benchmarks show 2.6M reads/sec

****Routing accuracy**** >90% correct domain routing JEPA gate decisions vs. Oracle

Consolidation quality >70% of GNN-predicted edges validated by Bonsai Bonsai accept rate on proposed edges

Forgetting accuracy <5% of pruned edges later needed Track retrieval failures on archived edges

****Context efficiency**** Graph-based context ≤50% the size of full-history context for equivalent recall Character count comparison

****Uncertainty calibration**** >80% precision on "I don't know" decisions Self-Model decisions vs. Oracle

Delegation efficiency >80% of queries handled by ≤8B model Model size distribution across queries

11. What This Architecture Enables

11.1 Any LLM Becomes a Memory-Augmented LLM

The architecture is a drop-in layer between the user and any language model. The LLM doesn't know about the memory system. It receives context and generates responses. GPT-4 today, Claude tomorrow, a local model next week — the subconscious doesn't care.

11.2 The Bidirectional Interface

The LLM can write back to the subconscious. Embedded in its response are memory commands — `[REMEMBER: this is important]`, `[LINK: Python, performance_issues, WaveDB]`, `[FORGET: the Postgres decision was superseded]`. These are stripped before the user sees the response and executed by the subconscious.

11.3 Cost-Optimal Intelligence

Most real-world queries are factual recall or basic synthesis. A 1-3B model handles 80% of queries. An 8B model handles 15%. A 70B+ model handles 5%. The routing is learned, not hardcoded. The system uses exactly as much intelligence as the task requires.

11.4 Domain Knowledge as Transferrable Graphs

You export your database domain graph. Your colleague imports it. Now their ponder engine knows everything yours knows about WaveDB — not because you fine-tuned their model, but because you shared a graph. The processing models are identical. The knowledge is what transferred.

11.5 The Subconscious Becomes the Primary Intelligence

As the Retrieval Gate improves, it handles an increasing fraction of cognitive work. It doesn't just route — it anticipates, it prepares, it pre-emptively retrieves. When you start typing "What was the throughp —", the gate has already activated the database domain, retrieved the Python async performance episodes, and loaded them into the SSM state. By the time you finish the question, the answer is already in awareness.

12. References

- Teyler, T.J. & DiScenna, P. (1986). The hippocampal memory indexing theory. *Behavioral Neuroscience*, 100(2), 147-154.
- Cowan, N. (1999). An embedded-processes model of working memory. In A. Miyake & P. Shah (Eds.), *Models of Working Memory*.
- McClelland, J.L., McNaughton, B.L., & O'Reilly, R.C. (1995). Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3), 419-457.
- Damasio, A.R. (1994). *Descartes' Error: Emotion, Reason, and the Human Brain*. — Somatic marker hypothesis, the neural basis of intuition.
- Gu, A. & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces.
- LeCun, Y. (2022). A path towards autonomous machine intelligence. — JEPA architecture.

- DeFelipe, J. (2011). The evolution of the brain, the human nature of cortical circuits, and intellectual creativity. *Frontiers in Neuroanatomy*, 5, 29.
 - WaveDB: github.com/vijayee/WaveDB
 - GLiNER2: github.com/fastino-ai/GLiNER2
 - GLiNER-Decoder: huggingface.co/knowledgator/gliner-decoder-base-v1.0
 - Memory-Traces: huggingface.co/datasets/Cossale/memory-traces
 - ConvoMem: github.com/SalesforceAIResearch/ConvoMem
 - EverMemBench: huggingface.co/datasets/EverMind-AI/EverMemBench-Dynamic
 - CloneMem: huggingface.co/datasets/ZhiyuZhangA/CloneMem
-

This architecture is designed to be built incrementally. Each phase produces a working system. Phase 1 alone – GLiNER2 extraction + WaveDB storage + basic graph retrieval – is a functional memory system that outperforms sliding-window context. Each subsequent phase adds capability without requiring redesign of earlier components. The architecture separates knowledge from processing, making domain expertise a transferrable, composable resource. The subconscious router learns from every interaction, becoming personalized over time. The ponder engine doesn't think – it ensures that when it's time to think, everything needed is already in awareness.